

Fourth Module

Marco Barberio

Master Degree in Computer Science

`m.barberio2@studenti.unipi.it`

Parallel and Distributed Systems: Paradigms and Models, Academic Year: 2025/2026

1 MPI distributed strategy

The MPI implementation preserves the same partitioned hash join structure used in the previous modules. The main difference is that, in this module, the input relations are distributed across multiple MPI processes, each running on a different node of the cluster. Therefore, before performing the local joins, records must be redistributed so that all records belonging to the same partition are assigned to the same MPI rank.

Each relation is initially generated in parallel by the MPI ranks. The total input is divided into contiguous ranges, and each rank generates only its local portion of the relation using the same input generation scheme adopted in the previous modules. The partition identifier of a record is computed using the same mapping function as in the sequential and shared-memory implementations.

The owner of a partition is determined by applying a modulo mapping over the number of MPI ranks. In particular, a partition with identifier `pid` is assigned to rank:

$$\text{owner} = \text{pid} \bmod \text{world_size}.$$

After locally classifying records by destination rank, the implementation uses `MPI_Alltoall` to exchange the number of records that must be sent to each rank. Then, `MPI_Alltoallv` is used to redistribute the actual records. This redistribution phase is performed independently for the two input relations R and S .

Once the redistribution has completed, each MPI process owns all records of R and S belonging to the partitions assigned to that rank. At this point, the same local partitioning and local join logic used in the previous modules can be applied independently by each rank. Finally, the local

results are combined using `MPI.Reduce`. The final output consists of the total number of matches, `checksum1`, and `checksum2`.

This strategy keeps the original algorithmic structure recognizable, but makes the cost of data movement explicit. In contrast with the OpenMP version, where all threads share the same memory space, the MPI version must pay the cost of moving records among ranks before the local joins can be executed.

2 Experimental Results

All experiments were conducted on the `spmcluster`. The number of partitions was fixed to $P = 1024$, and keys were generated in the range $[0, 100000)$. The MPI implementation was tested using 1, 2, 4, and 8 nodes, with one MPI rank per node. Therefore, the number of MPI ranks was equal to the number of nodes.

For each configuration, execution times were averaged over five seeds, from 0 to 4. Both uniform and skewed workloads were evaluated. For the skewed workload, 16 hot partitions were used, with 90% of the records assigned to keys mapping to those partitions.

For strong scaling, the input size was fixed to:

$$N_R = N_S = 10^7.$$

For weak scaling, the input size was increased proportionally to the number of MPI ranks, starting from:

$$N_R = N_S = 20000$$

with one rank. Therefore, the largest weak-scaling configuration used 8 ranks and:

$$N_R = N_S = 160000.$$

The experimental campaign was executed in batch mode. In particular, a single `srun` invocation was used for each node count. This reduced the overhead and instability associated with launching many short Slurm job steps, without changing the MPI configuration of each individual experiment.

Correctness was checked automatically for each executed configuration by comparing the final MPI output with the sequential baseline. In particular, the total number of matches, `checksum1`,

and `checksum2` were compared after each run. This validation step was kept separate from the measured execution time.

2.1 Strong Scaling

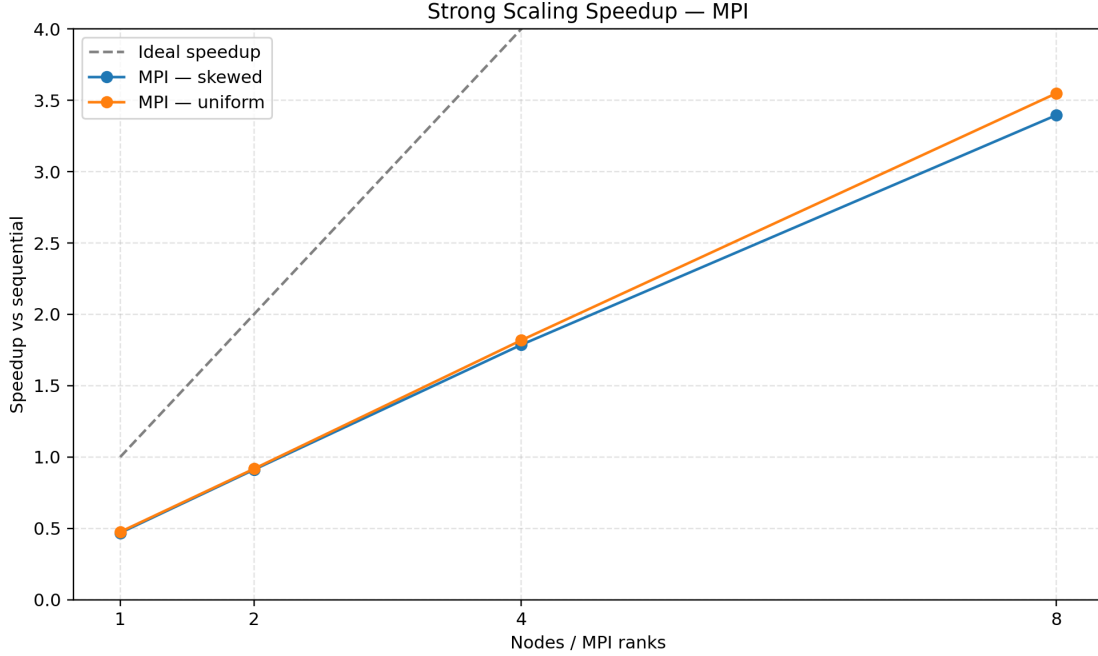


Figure 1: Strong scaling speedup of the MPI implementation under uniform and skewed workloads. The speedup is computed with respect to the sequential baseline.

Figure 1 reports the strong-scaling speedup of the MPI implementation. The speedup is computed with respect to the sequential baseline. The MPI version scales as the number of nodes increases, but remains below the ideal linear trend. On 8 nodes, it reaches approximately $3.53\times$ speedup on the uniform workload and approximately $3.46\times$ on the skewed workload.

This result is lower than the shared-memory implementations from the previous modules. In Module 2, the C++ thread-pool implementation, evaluated only on the uniform workload, reached approximately $5.3\times$ speedup at 8 threads. In Module 3, the OpenMP loop-based implementation reached approximately $5.3\times$ on the uniform workload and $5.2\times$ on the skewed workload at 8 threads, while the OpenMP task-based implementation reached approximately $4.9\times$ and $4.4\times$, respectively.

Overall, the strong-scaling trend is therefore different between shared-memory and distributed-

memory execution. Modules 2 and 3 scale better at the same degree of parallelism because all workers operate in the same address space. The MPI implementation, instead, must explicitly redistribute records among ranks before the local joins can be executed. This communication cost limits the strong-scaling speedup, even though the local computation decreases as the number of nodes grows.

2.2 Weak Scaling

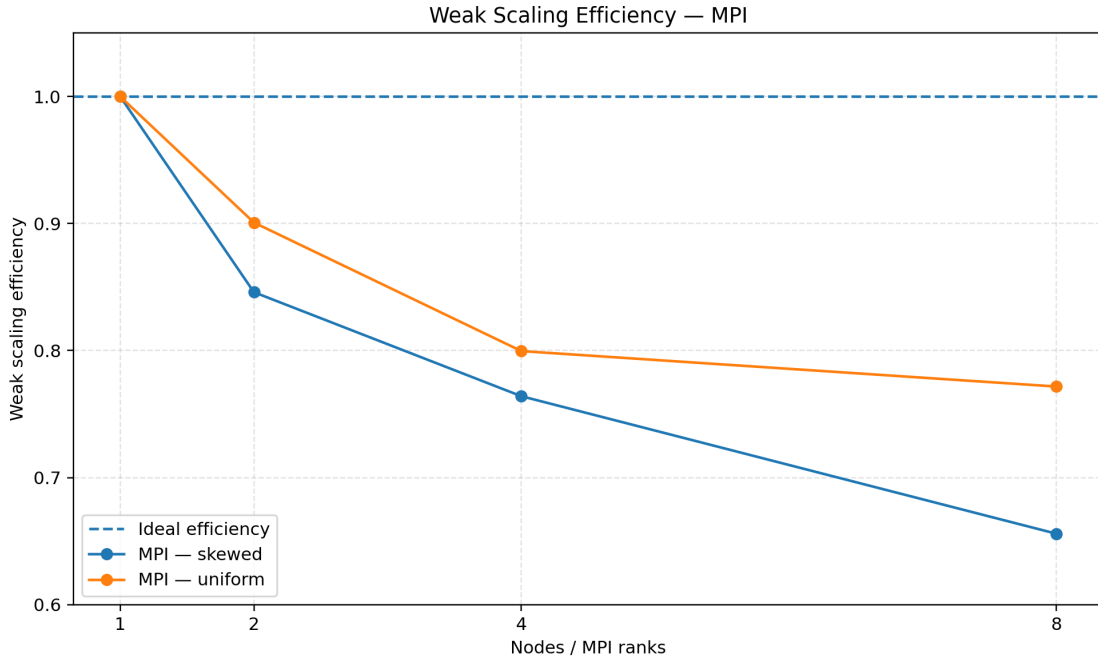


Figure 2: Weak scaling efficiency of the MPI implementation under uniform and skewed workloads.

Figure 2 reports the weak-scaling efficiency of the MPI implementation. The MPI implementation maintains a reasonably stable efficiency on the uniform workload, reaching approximately 0.77 on 8 nodes. On the skewed workload, the efficiency decreases more clearly, reaching approximately 0.65 on 8 nodes.

Compared with the previous shared-memory implementations, the MPI weak-scaling behavior is competitive. In Module 2, the C++ thread-pool implementation reached approximately 0.71 efficiency at 8 threads on the uniform workload. In Module 3, the OpenMP loop-based implementation reached approximately 0.63 on the uniform workload and 0.51 on the skewed workload at 8 threads, while the OpenMP task-based implementation reached approximately 0.55 and 0.40,

respectively.

Therefore, although the MPI implementation is less effective in strong scaling, it behaves well in weak scaling. This is because increasing the number of nodes also increases the available distributed memory and computational resources. However, the efficiency still decreases as the number of nodes grows, especially under skewness, because the redistribution phase becomes more expensive and the work is less evenly balanced among ranks.

2.3 Communication and local computation

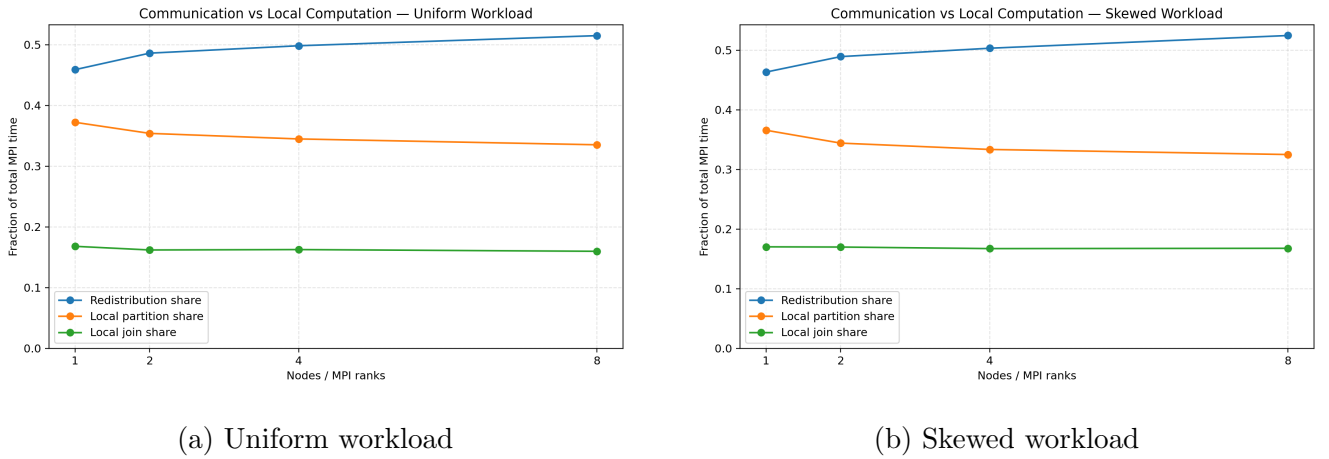


Figure 3: Fraction of the total MPI execution time spent in redistribution, local partitioning, and local join phases.

Figure 3 reports the fraction of the total MPI time spent in three main components: redistribution, local partitioning, and local join. This plot highlights the main difference between the distributed-memory implementation and the shared-memory implementations from the previous modules.

The redistribution phase is the dominant component of the MPI execution. It accounts for almost half of the total time already with one node, and its relative contribution increases as the number of nodes grows. On 8 nodes, the redistribution phase represents slightly more than half of the total MPI execution time.

At the same time, the fraction of time spent in local partitioning decreases slightly, while the local join phase remains relatively stable and accounts for a smaller portion of the total time. This indicates that the main bottleneck of the MPI implementation is not the local join itself, but the cost of moving records to the ranks responsible for their partitions.

The same trend appears for both uniform and skewed workloads. Therefore, the communication overhead is a structural cost of the adopted MPI strategy, not only a consequence of skewness.

2.4 Phase Breakdown

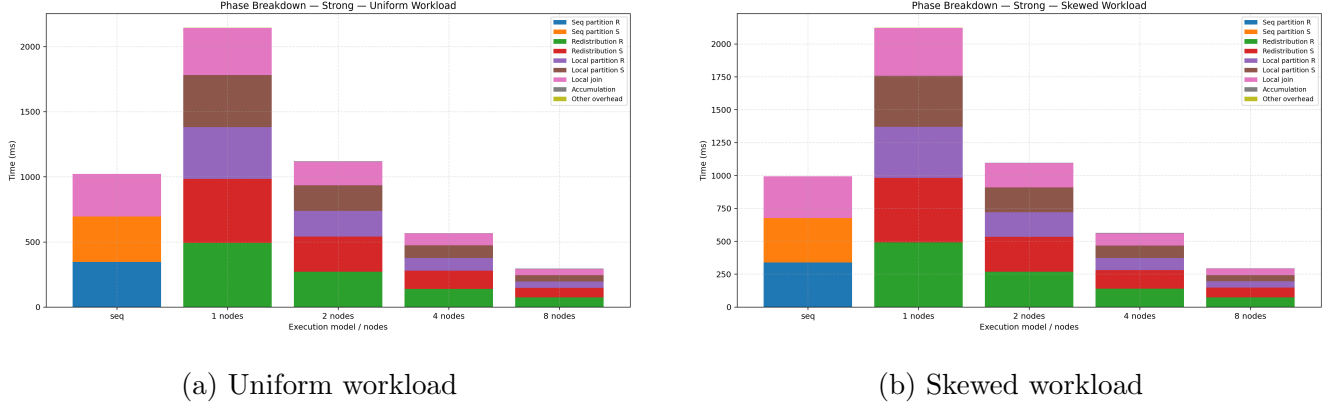


Figure 4: Execution time breakdown by phase for the sequential baseline and for the MPI implementation with 1, 2, 4, and 8 nodes.

Figure 4 provides a detailed execution-time breakdown for the strong-scaling experiment. The first bar corresponds to the sequential baseline, while the other bars correspond to the MPI implementation with 1, 2, 4, and 8 nodes.

The one-node MPI execution is slower than the sequential baseline because it still executes the distributed pipeline, including redistribution and local partitioning, without benefiting from inter-node parallelism. As the number of nodes increases, the total execution time decreases, but the redistribution components remain substantial in all configurations. This confirms that the strong-scaling speedup is mainly limited by communication overhead rather than by the local join phase alone.

The same overall behavior appears for both uniform and skewed workloads, with only slightly higher costs under skewness.

3 Discussion and Comparison with Previous Modules

The MPI implementation preserves the same high-level partitioned hash join algorithm studied in Modules 2 and 3, but it targets a different execution model. Module 2 used an explicit C++

thread-pool implementation, while Module 3 expressed the same shared-memory parallelism using OpenMP loop-level and task-based constructs. In both cases, all workers operated within a shared address space. Therefore, the main performance issues were memory bandwidth, scheduling overhead, synchronization, and load imbalance among partitions.

In Module 4, instead, each MPI rank owns a private address space. As a result, records belonging to the same partition must be explicitly moved to the rank responsible for that partition before the local join can be executed. This introduces a communication and redistribution phase that is not present in the same form in the shared-memory implementations.

Overall, the comparison with Modules 2 and 3 shows that the distributed-memory implementation changes the main performance bottleneck. In the shared-memory versions, scalability was mainly limited by memory-bound phases such as scatter, coordination overhead, and load imbalance under skewness. In the MPI version, these issues are still present, but the dominant additional cost is explicit data movement among ranks. Therefore, MPI can exploit multiple nodes and distributed memory, but its benefit depends on whether the reduction in local computation is large enough to compensate for redistribution and communication overhead.